DT081A - Signal and Image Processing

Lab 3 Report — Image Processing

Yacoub Dawli — yada2100@student.miun.se

24 October 2026

1 Introduction

This lab covered pixel transforms, colour reduction, filtering, Fourier analysis, histograms and morphology, comparing image changes, time and memory use.

2 Experimental Setup

The verified experiments used Python, NumPy, OpenCV 5.0.0 and Matplotlib, with the course 512×512 peppers TIFF (`ManWoman.png` was unavailable; other images are permitted). BGR was converted to RGB for display, with grayscale and synthetic tests where needed. No experiments were rerun.

3 Results and Analysis

3.1 Gamma Correction

Q1.1. The formula $255(I/255)^{\gamma}$, at $\gamma = 2.0$, darkened the three inputs (Table 1); `uint8` drops fractional parts.

Table 1: Gamma calculation, $\gamma = 2.0$.

Input	Before integer conversion	`uint8` output
64	16.063	16
128	64.251	64
192	144.565	144

Q1.2. At $\gamma = 2.2$, mean intensity fell from 120.22 to 61.00. The image darkened; its histogram moved lower, with integer-conversion gaps (Figure 1). This relates to unequal visual sensitivity, but is not an exact logarithmic eye model. The lab uses γ, not camera pre-correction's $1/\gamma$.

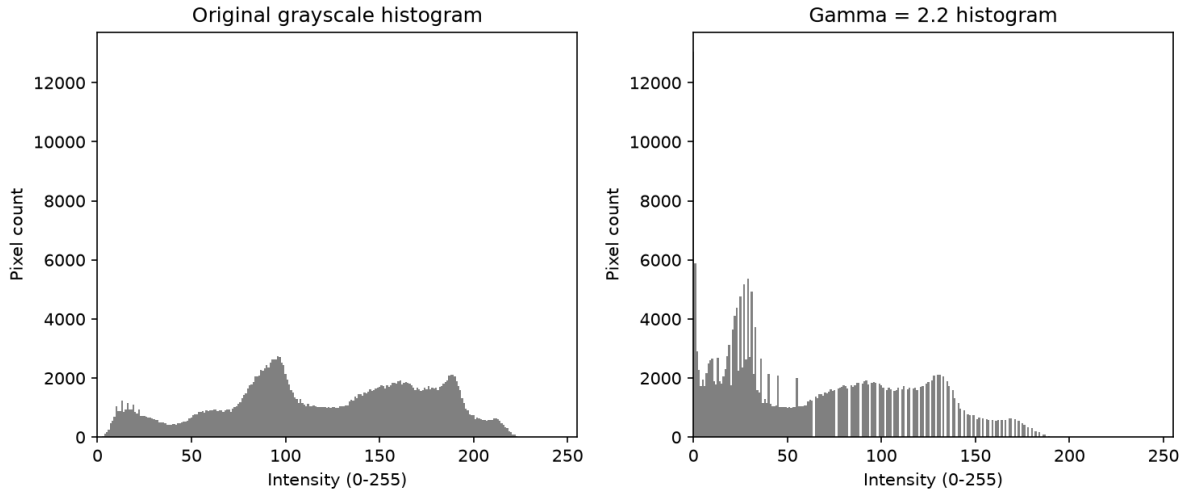


Figure 1: Q1.2: original and gamma-2.2 histograms from the same experiment.

3.2 Colorspaces

Q2.1. OpenCV orders channels Y, Cr, Cb; $Y \approx 0.299R + 0.587G + 0.114B$. All minima were 0; Y/R/G/B maxima were 228/231/237/229 and standard deviations 53.88/45.17/75.05/44.29. G was widest, then Y; R had many high values, B mostly low. Y gives one brightness channel for edge detection, but can miss equal-brightness colour boundaries.

Q2.2. Y stayed unchanged; both chroma dimensions were reduced by factors 2, 4 and 8, using area reduction and linear reconstruction. Factor 2 gives $3N \to N + N/4 + N/4 = 1.5N$: 786,432 to 393,216 bytes, saving 50%. Factors 4/8 need 294,912/270,336 bytes, saving 62.5/65.625% before reconstruction, not compressed files. Factor 2 looked close; 4 softened colour boundaries; 8 blended colours more, while Y detail remained.

3.3 Quantization and Sampling

Q3.1. RGB integer division by 16 gave 16 possible levels, displayed as $0, 16, \ldots, 240$. Only 15 were used: no input reached 240. Colours became stepped and darker. Packed 4-bit storage gives 2:1, or 50% saving: 393,216 versus 786,432 bytes. The actual `uint8` array stayed at 786,432 bytes.

Q3.2. Grayscale division by 32 groups 32 intensities into each of eight levels $0, 32, \ldots, 224$. All eight were used; the last held 29 pixels. Both histograms count 262,144 pixels (Figure 2). Grouping caused banding and lost shading/texture, although outlines remained.

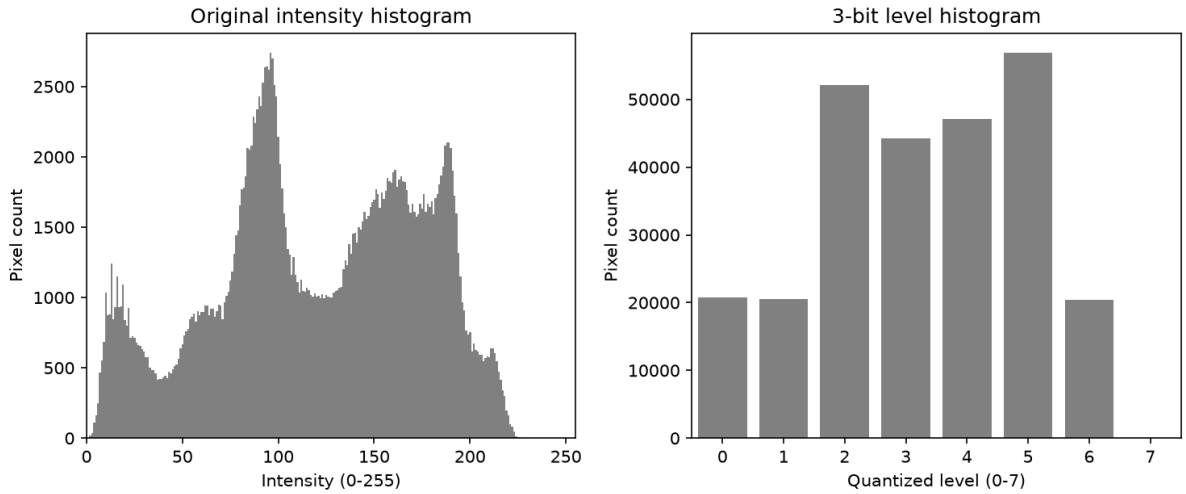


Figure 2: Q3.2: original and 3-bit histograms; count scales are labelled separately.

3.4 Convolution and Edge Detection

Q4.1. Matching $\sigma = 1$ Gaussian kernels used float64 and `BORDER_REFLECT_101`, after warm-up (30 runs). Recorded 2D/separable means in ms were: 5×5: 5.420/1.429; 11×11: 9.880/10.455; 21×21: 23.481/24.022. Loading and plotting were excluded. Separable was faster only at 5×5; 2D was slightly faster at 11 and 21. Reducing k^2 weights to about $2k$ can help large kernels, but library routines affect speed. Outputs looked alike, differing by at most about 0.000002 intensity units.

Q4.2. The 3×3 Sobel function used `dx=0, dy=1`. Absolute top-to-bottom changes highlighted horizontal boundaries and weakened vertical sides. With `dx=1, dy=0`, vertical edges stood out; curves appeared in both.

Q4.3. Signed derivatives gave $G = \sqrt{G_x^2 + G_y^2}$, with a copy normalized to 0–255. Combining directions completed outlines/stems; brighter means stronger edges (Figure 8, right).

3.5 Matrix Operations

Q5.1. Stacking, applying $1.2(I-128)+128+20$, clipping to 0–255 and reshaping gave identical brighter NumPy/loop images, with clipped highlights. NumPy was 229.82 times faster using unrounded times (NumPy 2.040 ms; loops 468.915 ms, 5 runs), avoiding Python operations per pixel. Timing included calculation, output creation and conversion.

Q5.2. The 262,144-value vector gives H $262{,}144^2 = 68{,}719{,}476{,}736$ elements. Float32 needs 274,877,906,944 bytes (256 GiB); float64 549,755,813,888 bytes (512 GiB), with GiB $= 1024^3$ bytes. H was not allocated. Most entries would be zero; storing nine weights and filtering locally is more practical.

3.6 Pixel-Based Processing

Q6.1. Direct and 256-entry lookup outputs were identical `uint8`, range 0–240. Lookup was 3.84 times faster (direct 9.469 ms; lookup 2.468 ms), reusing logarithms. Its separate 0.539 ms construction was excluded; warmed-up applications were averaged over 30 calls.

Q6.2. Two log halves joined at 127.5, strength 4 (Figure 3). The middle is steeper, the ends flatter: input gaps 32 gave output gaps 17 (0–32), 64 (112–144), 18 (223–255). Middle shading separated; shadows/highlights compressed. Dark gaps darkened and light areas brightened. Intensity order and endpoints 0 and 255 stayed unchanged.

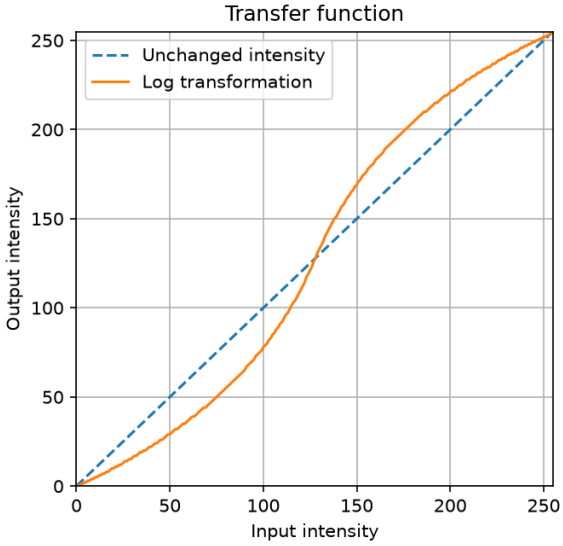


Figure 3: Q6.2: logarithmic transfer curve; dashed line is unchanged intensity.

3.7 Elementary Images

Q7.1. The teacher's outer products made 16×16 images: Ramp×Ramp gave two-direction gradients; Step×Ramp a boundary plus gradient; Step×Step a sharp lower-right block; Step×Sine repeating lower-half columns. On the shared -1 to 1 scale, zero was middle gray.

Q7.2. Least squares fitted Step×Ramp, Step×Step and Step×Sine to the 8×8 square (rows 8–15, columns 4–11, zero-based). Weights: 0.156058, 0.812731, 0.688038; MSE 0.107400 on 0–1; raw range 0.000000–0.895961. The horizontal boundary matched, but uneven interiors, wrong sides and outside bands gave a coarse fit (Figure 4).

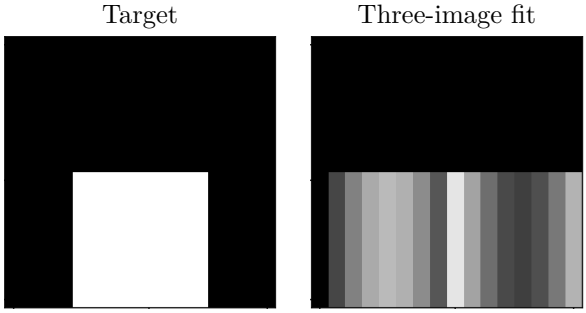


Figure 4: Q7.2: target and raw fit. Black 0 to white 1 in both.

3.8 Transforms and Frequency Domain

Q8.1. The display used `log1p(abs(fftshift(fft2(gray))))`. The bright centre holds average brightness and slow changes; faster changes lie farther out. Horizontal/vertical streaks include border effects. Vertical edges contribute horizontal frequencies and vice versa; curves contribute several directions. FFT does not locate individual edges.

Q8.2. The circular low-pass kept radii below 40 bins in the 512×512 grid. Inverse shifting/FFT gave its real part. Main shapes stayed; detail weakened, boundaries softened and the sharp cutoff caused ringing (Figure 5). Raw range: -17.078 to 243.781; negatives display black without changing data.

Q8.3. Floating-point original-minus-low-pass kept outlines, stems and detail, with smooth areas near zero (Figure 5). Raw range: -155.400 to 113.396; display ± 155.400, zero middle gray. Ringing remained. Low-pass plus high-pass recovered the original with maximum error 0; the inverse FFT's largest imaginary part was 6.172×10^{-14}.



Figure 5: Q8.2–8.3: original/low-pass use 0–255; high-pass uses ± 155.400, zero middle gray.

3.9 Histograms and Image Enhancement

Q9.1. The normalized histogram sums to 1 (Figure 6): 76.43% of pixels were 64–191, the peak was 96, another group 150–190, maximum 228. Cumulative values: 52.51% at 127, 92.19% at 191, reaching 1 at 228. These show intensity amounts, not positions. `[0,256]` includes all levels because OpenCV excludes the upper limit.

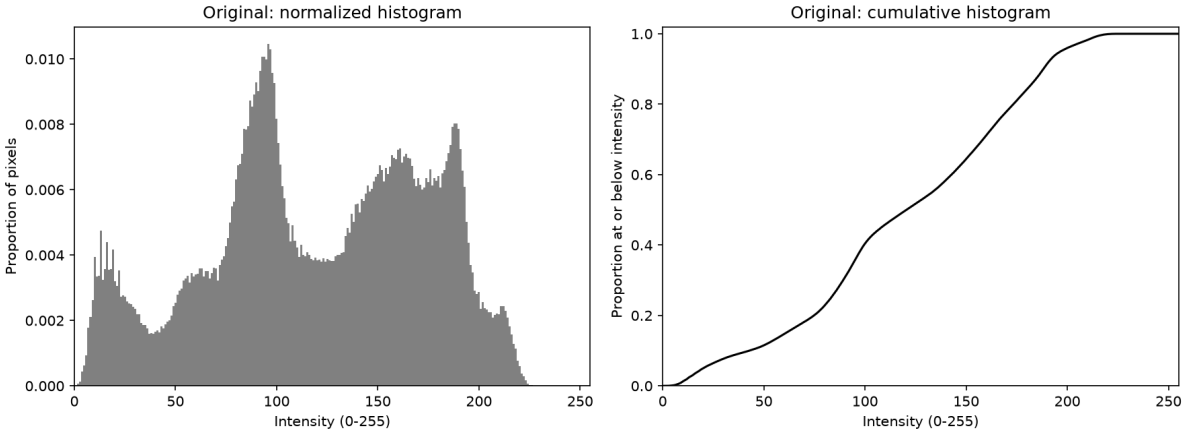


Figure 6: Q9.1: normalized histogram and cumulative histogram (CDF).

Q9.2. Floating-point $128 + 0.5(I - 128)$ shrank 0–228 to 64–178. $T(r) = 255\,\mathrm{CDF}(r)$, converted to `uint8`, restored 0–255; OpenCV's `equalizeHist()` did too (Figure 7). Details became clearer than the flat input; gaps/highlights were stronger than originally, with more grain. Maximum difference: 1 level; mean: 0.443756. OpenCV subtracts the first nonzero cumulative value and rounds; this method truncates the lab's formula.

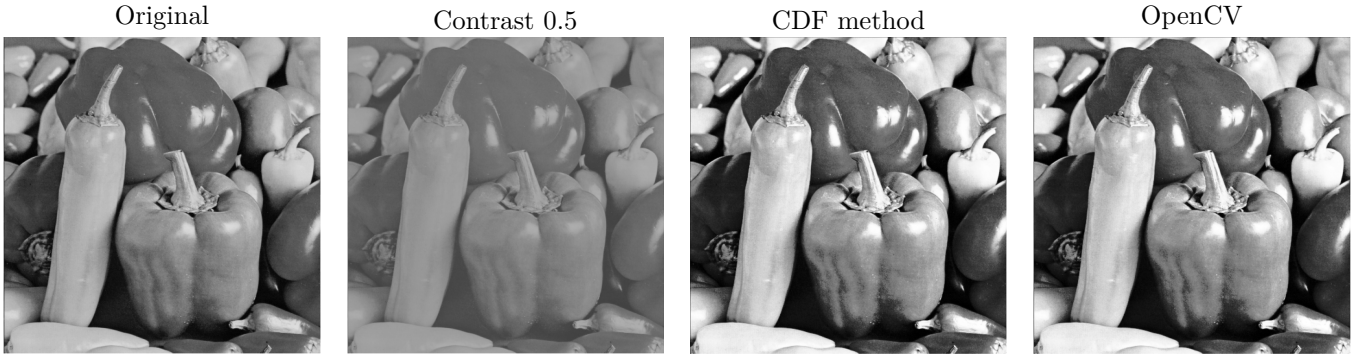


Figure 7: Q9.2: original, reduced contrast and two equalizations. All displays use 0–255.

3.10 Noise and Noise Reduction

Q10.1. Seed 42 gave one zero-mean Gaussian sample, scaled and added as floats, then clipped before integer conversion. Median 3×3, noise $\sigma = 25$: PSNR 20.318 to 26.829 dB. Bilateral (diameter 9, colour/spatial sigmas 75/75), $\sigma = 35$: 17.519 to 26.888 dB. The 0.059 dB gap is close, but different input noise prevents same-input ranking. Bilateral trials $\sigma = 25, 30, 35$: 28.860, 28.004, 26.888 dB. Both reduced grain; bilateral was smoother but softened stems. Similar PSNR did not mean identical appearance.

Q10.2. Both 3×3 filters received the same noisy input. PSNR in dB (input/mean/median; mean-minus-median): Gaussian $\sigma = 25$, 20.318/27.237/26.829 (+0.409); salt-and-pepper 4%, 19.296/26.516/34.035 (−7.519). Salt-and-pepper probability was 4% total: 2% black, 2% white, no overlap; 3.986% selected. Mean gave slightly higher PSNR for Gaussian noise but blurred edges. Median removed isolated extremes; mean spread them into gray marks. `cv2.PSNR()` used the clean image; differences use unrounded values.

3.11 Morphological Filtering

Q11.1. Two 128×128 binary tests used separated shapes: A had a circle, rectangle and triangle; B added a different-size circle. Each had three white specks and a 2×2 hole. Opening with a 3×3 cross removed specks; closing with a 5×5 ellipse filled holes. Eight-connected labels minus background gave expected/detected 3/3 and 4/4. Corners rounded slightly; touching objects would count together.

Q11.2. Dilation minus erosion with the cross showed boundaries and more grain; Sobel was smoother (Figure 8). Raw ranges 0–204 and 0–812.857 were separately normalized to 0–255: display brightness does not compare raw strength. Morphology suits binary shapes and kernel-dependent boundaries; Sobel suits grayscale changes and directional derivatives. Magnitude gives strength, not direction.

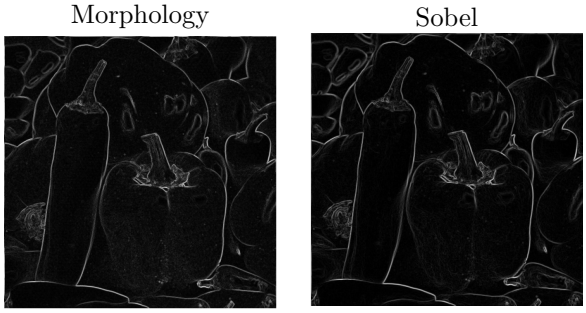


Figure 8: Q11.2 and Q4.3: gradients, separately normalized to 0–255.

4 Discussion

Vectorization and lookup saved time without changing outputs. Gaussian speed depended on routines; one run gives no general ranking. Filters traded noise against detail and depended on noise type. Equalization exposed grain. The coarse fit and touching-object limit show why settings and representation matter.

5 Conclusion

The experiments linked pixel values, colour, detail and frequency content. Numerical results supported the image comparisons and explained time, memory and noise trade-offs.

Appendix: Code Snippets

FFT filtering. The circular mask keeps low frequencies; the inverse FFT returns the smooth image.

```
filtered_fft = fshift * lowpass_mask
lowpass_complex = np.fft.ifft2(np.fft.ifftshift(filtered_fft))
```